Creational Design Pattern Report

Oct 16, 2026 · @Alibi Auyespek

1. Введение

Порождающие (creational) паттерны отвечают на вопрос «как создать объект». Они отделяют код, который *использует* объект, от кода, который его *создаёт*. Благодаря этому можно менять тип и способ создания объектов, не трогая остальную программу.

В классическом каталоге GoF («Gang of Four», 1994) пять порождающих паттернов: **Singleton**, **Factory Method**, **Abstract Factory**, **Builder** и **Prototype**. В отчёте каждый разобран по схеме «проблема → решение → код на Python». В конце показано практическое применение Singleton и Factory Method в веб-приложении на Flask (Lab 6).

Технологии: Python 3, модули `abc` и `copy`; для практики — Flask, Flask-SQLAlchemy, SQLite.

2. Singleton (Одиночка)

Суть: у класса есть только один объект, и к нему есть глобальная точка доступа.

Проблема. Некоторые объекты должны быть в программе в единственном экземпляре: настройки, подключение к базе данных, логгер. Если каждый модуль создаст свой объект, появятся лишние соединения и расхождение данных.

Решение. Класс сам контролирует создание объекта. В Python для этого переопределяют `__new__`: он вызывается до `__init__` и решает, создать новый объект или вернуть уже существующий.

```
class Config:
    instance = None

    def __new__(cls):
        if cls.instance is None:
            cls.instance = super().__new__(cls)
        return cls.instance

a = Config()
b = Config()
print(a is b)   # True — это один и тот же объект
```

Плюсы: гарантированно один объект, ленивое создание (только при первом вызове). **Минусы:** это скрытое глобальное состояние, его сложнее подменить в тестах; в многопоточной программе нужна блокировка (`threading.Lock`).

3. Factory Method (Фабричный метод)

Суть: класс объявляет метод для создания объекта, а какой именно объект создать, решают подклассы.

Проблема. Код доставки, в котором жёстко написано `Truck()` , нельзя переиспользовать для морской доставки: придётся писать `if/elif` и править код при каждом новом виде транспорта.

Решение. Создание объекта выносится в отдельный метод `create_transport()` . Каждый создатель сам решает, какой транспорт вернуть, а клиент работает с общим интерфейсом `deliver()` .

```
class Truck:
    def deliver(self):
        return "Везём грузовиком"

class Ship:
    def deliver(self):
        return "Везём кораблём"

class RoadDelivery:
    def create_transport(self):   # фабричный метод
        return Truck()

class SeaDelivery:
    def create_transport(self):
        return Ship()

print(RoadDelivery().create_transport().deliver())  # Везём грузовиком
print(SeaDelivery().create_transport().deliver())   # Везём кораблём
```

Плюсы: клиент не зависит от конкретных классов; новый тип добавляется новым классом без правки старого кода (принцип Open/Closed). **Минусы:** растёт число классов.

4. Abstract Factory (Абстрактная фабрика)

Суть: фабрика создаёт целое **семейство** связанных объектов, которые подходят друг к другу.

Проблема. В интерфейсе со светлой и тёмной темой кнопка и текст должны быть из одной темы. Если создавать их по отдельности, легко получить тёмную кнопку на светлом тексте.

Решение. Каждая тема — это фабрика с одинаковым набором методов (`button()`, `text()`). Клиент выбирает одну фабрику и получает совместимый набор элементов.

```
class LightTheme:
    def button(self):
        return "Светлая кнопка"

    def text(self):
        return "Светлый текст"

class DarkTheme:
    def button(self):
        return "Тёмная кнопка"

    def text(self):
        return "Тёмный текст"

for theme in [LightTheme(), DarkTheme()]:
    print(theme.button(), "и", theme.text())
```

Отличие от Factory Method: фабричный метод создаёт **один** объект, абстрактная фабрика — **набор** связанных объектов. **Минус:** чтобы добавить новый вид элемента (например, `checkbox()`), придётся изменить все фабрики.

5. Builder (Строитель)

Суть: сложный объект собирается по шагам, а готовый результат возвращает метод `build()`.

Проблема. У пиццы много необязательных параметров: размер, сыр, грибы, соус. Конструктор вида `Pizza(30, True, False, True, ...)` неудобен и легко путает порядок аргументов.

Решение. Строитель даёт отдельный метод на каждый шаг. Каждый метод возвращает `self`, поэтому вызовы можно записать цепочкой.

```
class PizzaBuilder:
    def __init__(self):
        self.size = 0
        self.toppings = []

    def set_size(self, size):
        self.size = size
        return self

    def add(self, topping):
        self.toppings.append(topping)
        return self

    def build(self):
        return f"Пицца {self.size} см: {', '.join(self.toppings)}"

print(PizzaBuilder().set_size(30).add("сыр").add("грибы").build())
# Пицца 30 см: сыр, грибы
```

Плюсы: читаемый код создания, можно собирать разные варианты одного объекта. **Минус:** нужен отдельный класс-строитель даже для простых случаев.

6. Prototype (Прототип)

Суть: новый объект создаётся копированием уже существующего, а не с нуля.

Проблема. Иногда нужен объект, почти как готовый (шаблон договора), и собирать его заново долго. Простое присваивание `doc2 = doc` не поможет: обе переменные будут указывать на один объект.

Решение. Объект сам умеет себя копировать через метод `clone()`. `copy.deepcopy` делает полную копию, включая вложенные списки.

```
import copy

class Document:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

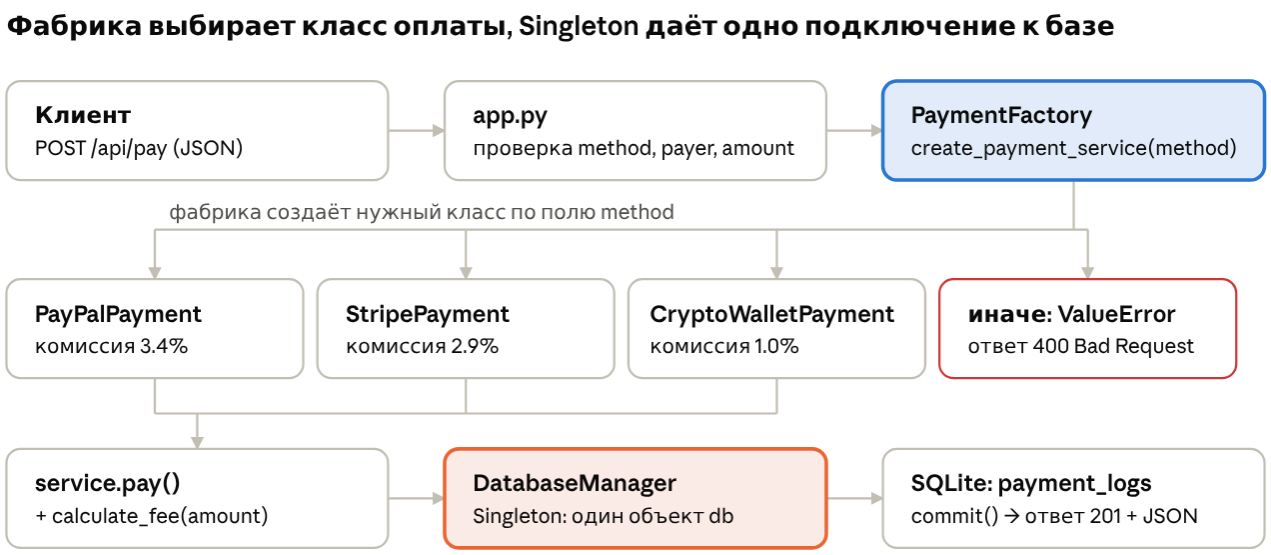
    def clone(self):
        return copy.deepcopy(self)

doc = Document("Договор", ["Стр. 1"])
doc2 = doc.clone()
doc2.pages.append("Стр. 2")
print(doc.pages, doc2.pages)   # ['Стр. 1'] ['Стр. 1', 'Стр. 2']
```

Оригинал не изменился — копия полностью независима. При `copy.copy` (поверхностная копия) список `pages` был бы общим, и оригинал тоже получил бы «Стр. 2».

7. Практика: Singleton и Factory Method в Lab 6

В Lab 6 оба паттерна применены в REST API платёжного агрегатора (вариант 2: PayPal, Stripe, Crypto Wallet) на Flask и SQLAlchemy. Клиент отправляет способ оплаты, фабрика создаёт нужный сервис, а платёж сохраняется через единственный `DatabaseManager`.



путь запроса POST /api/pay · 2 паттерна

Маршрут `app.py` не знает конкретных классов оплаты: он получает объект от фабрики и вызывает у него `pay()`.

Factory Method (`factory.py`):

```
class PaymentFactory:
    _services = {
        "paypal": PayPalPayment,
        "stripe": StripePayment,
        "crypto": CryptoWalletPayment,
    }

    @staticmethod
    def create_payment_service(method):
        service_class =
PaymentFactory._services.get(str(method).strip().lower())
        if service_class is None:
            raise ValueError(f"Unsupported payment method: '{method}'")
        return service_class()
```

Singleton (`database.py`):

```
class DatabaseManager:
    _instance = None

    def __new__(cls, app=None):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.db = SQLAlchemy()
            cls._instance._initialized_app = None
        if app is not None:
            cls._instance.init_app(app)
        return cls._instance
```

Результат проверки:

Запрос	Созданный класс	Сумма, $	Комиссия, $	Ответ
`method: paypal`	`PayPalPayment`	100	3.40	201
`method: stripe`	`StripePayment`	250.50	7.26	201
`method: crypto`	`CryptoWalletPayment`	1000	10.00	201
`method: applepay`	— (ValueError)	50	—	400

`GET /api/singleton-check` возвращает `same_object: true` : у всех вызовов `DatabaseManager()` одинаковый `id()` .

В шаблоне из задания была ошибка: проверка `if app:` стояла внутри `if cls._instance is None`. Поэтому второй вызов `DatabaseManager(app)` не подключал базу. В решении проверка вынесена наружу.

8. Сравнение и вывод

Паттерн	Когда применять	Пример в коде
Singleton	объект должен быть один на всю программу	`Config()`, `DatabaseManager()`
Factory Method	тип объекта известен только во время работы	`RoadDelivery` → `Truck`, `PaymentFactory`
Abstract Factory	нужен совместимый набор объектов	`DarkTheme` → кнопка + текст
Builder	у объекта много необязательных частей	`PizzaBuilder().set_size(30).add("сыр")`
Prototype	нужна независимая копия готового объекта	`doc.clone()` через `deepcopy`

Вывод. Порождающие паттерны прячут детали создания объектов, и код зависит от абстракций, а не от конкретных классов. На практике в Lab 6 Singleton дал одно подключение к базе на всё приложение, а Factory Method позволил добавлять новые способы оплаты без изменения API. Применять паттерны стоит только там, где есть соответствующая проблема: иначе они лишь усложняют код.